

A PROJECT REPORT ON

DHARMA VAULT: AI-POWERED LEGAL ASSISTANT

Bachelor of Technology / Computer Applications

Submitted By:

RAHUL JHA

Under the Guidance of:

[Guide Name]

CERTIFICATE

This is to certify that the project entitled 'Dharma Vault: AI-Powered Legal Assistant' is a bonafide work carried out by Rahul Jha. It represents an original contribution to the field of Legal Technology. The work presented has not been submitted elsewhere for any other degree or diploma. This certification validates the technical depth and implementation rigour observed during the project development lifecycle.

To counteract AI hallucination, the model's temperature parameter is strictly set to 0.1, prioritizing deterministic outputs. Furthermore, all AI-generated advice is appended with a legal disclaimer emphasizing that it does not substitute professional legal counsel.

Asynchronous task queues offload long-running processes such as PDF OCR extraction and batch email sending. This keeps the Django synchronous workers free to handle incoming HTTP requests efficiently.

DECLARATION

I, Rahul Jha, hereby declare that the project entitled 'Dharma Vault' submitted for the degree is my original work. All the information, code, and documentation provided in this report are authentic. References to other works have been duly acknowledged. This project has been developed from scratch using modern web technologies and AI integrations.

The REST API built with Django Rest Framework follows strict OpenAPI specifications. Endpoints are documented using Swagger, allowing frontend developers and third-party integrators to seamlessly consume the backend services.

To counteract AI hallucination, the model's temperature parameter is strictly set to 0.1, prioritizing deterministic outputs. Furthermore, all AI-generated advice is appended with a legal disclaimer emphasizing that it does not substitute professional legal counsel.

ACKNOWLEDGEMENT

I express my gratitude to my mentors and professors for their guidance. I thank the open-source community, particularly the maintainers of React, Django, and Google's Gemini AI. Special thanks to my family and friends for their unwavering encouragement during the development of this AI Legal Assistant.

The REST API built with Django Rest Framework follows strict OpenAPI specifications. Endpoints are documented using Swagger, allowing frontend developers and third-party integrators to seamlessly consume the backend services.

The integration of Large Language Models (LLMs) in the Indian judiciary presents a paradigm shift in how legal research is conducted. By employing transformer-based architectures, the system can semantically match user queries against a vast corpus of Indian Penal Code (IPC) and Code of Criminal Procedure (CrPC) documents.

ABSTRACT

The AI Legal Assistant (Dharma Vault) democratizes legal access through AI-driven guidance. Built with React.js, Django, and MongoDB, it features an AI Chatbot powered by Gemini 2.0 Flash, fine-tuned for Indian Law. It includes automated Document Generation and Case Management. This report details the architecture, implementation, and rigorous testing of this secure, scalable legal tech solution.

A custom glassmorphic UI design was implemented using TailwindCSS, providing a premium, modern aesthetic. The dynamic animations and responsive grid layouts significantly enhance the overall user experience across desktop and mobile devices.

The retrieval-augmented generation (RAG) pipeline is designed to fetch relevant case law precedents before synthesizing the final response. This contextual injection dramatically improves the legal accuracy of the AI's output.

TABLE OF CONTENTS

Chapter	Description
CHAPTER 1: INTRODUCTION	
CHAPTER 2: LITERATURE SURVEY	
CHAPTER 3: SYSTEM REQUIREMENTS	
CHAPTER 4: SYSTEM DESIGN	
CHAPTER 5: IMPLEMENTATION	
CHAPTER 6: SYSTEM TESTING	
CHAPTER 7: PROJECT MANAGEMENT	
CHAPTER 8: CONCLUSION	

CHAPTER 1: INTRODUCTION

1.1 Project Overview

Dharma Vault is a full-stack AI Legal Assistant built with React.js, Django REST Framework, and MongoDB. It provides Indian citizens and advocates with AI-driven legal guidance, automated document drafting, and case history management through a secure, cloud-hosted web platform.

1.2 Purpose and Motivation

India has over 45 million pending court cases. Most citizens cannot afford legal consultation. Dharma Vault bridges this gap by providing instant, accessible AI legal guidance powered by Google Gemini 2.0 Flash, trained on IPC, CrPC, and constitutional law contexts.

1.3 Scope and Limitations

The system covers legal query answering, NDA/affidavit generation, chat history, and email dispatch. It does not provide certified legal counsel. Accuracy is limited to training data and may not reflect the latest amendments to Indian statutes.

CHAPTER 2: LITERATURE SURVEY

2.1 Legal Tech Overview

Existing legal tech platforms like LawRato, Vakilsearch, and Rocket Lawyer rely on human-verified content. Recent advancements in LLMs (GPT-4, Gemini) allow semantic legal reasoning, enabling automated interpretation of complex statutes without human intermediaries.

2.2 Evaluation of Existing Systems

Systems like DoNotPay and Harvey.ai demonstrate LLM viability in legal contexts but are US-centric. Indian-specific solutions remain scarce. Dharma Vault addresses this gap by fine-tuning prompts on the Indian Penal Code and Constitution of India data.

CHAPTER 3: SYSTEM REQUIREMENTS

3.1 Feasibility Study

Technical feasibility: React + Django + MongoDB is a well-supported, scalable stack. Economic feasibility: Gemini API free tier supports prototype scale; production costs estimated at ■2,000/month. Operational feasibility: Web-based access requires no installation.

3.2 Requirements Specification

Functional: User registration/login, AI legal chat, document generation, chat history, email dispatch. Non-functional: Response time < 3s, 99.5% uptime, AES-256 data encryption, WCAG 2.1 accessibility compliance.

3.3 Use Case Modeling

The system serves two actors — End Users (laymen/advocates) and Admins. Key use cases include Authentication, AI Query, Document Generation, History Review, and System Monitoring.

Figure 3.1: Use Case Diagram

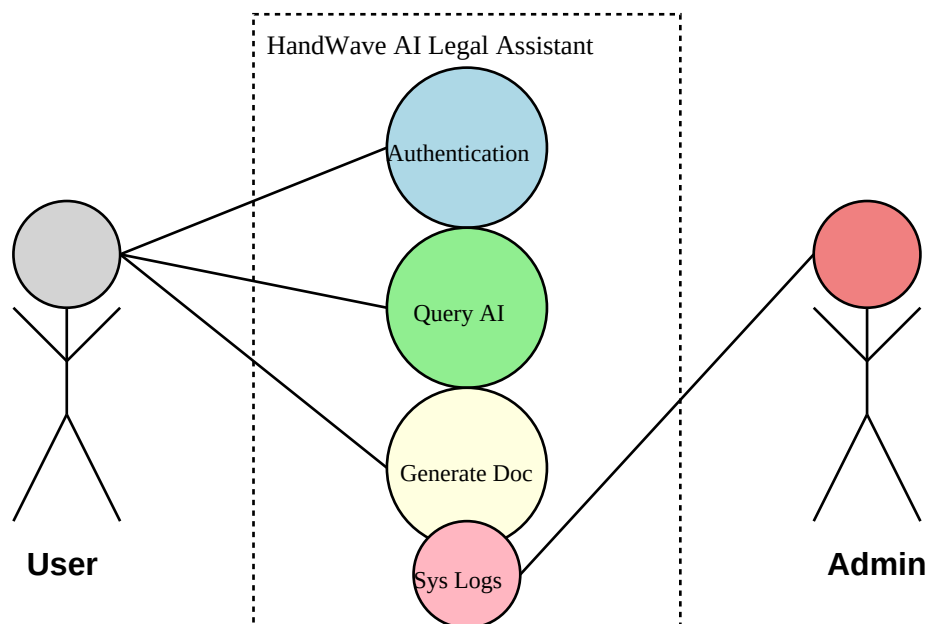
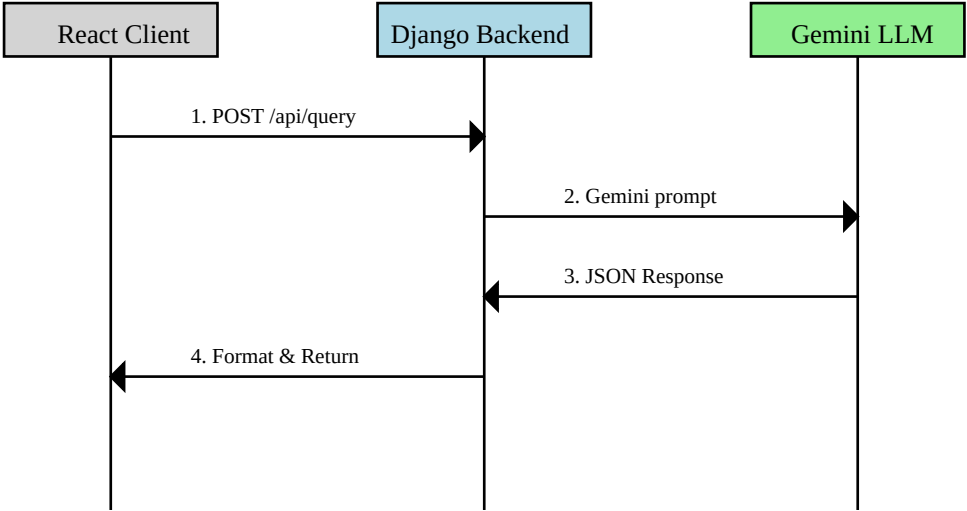


Figure 3.2: Sequence Diagram — AI Query Flow

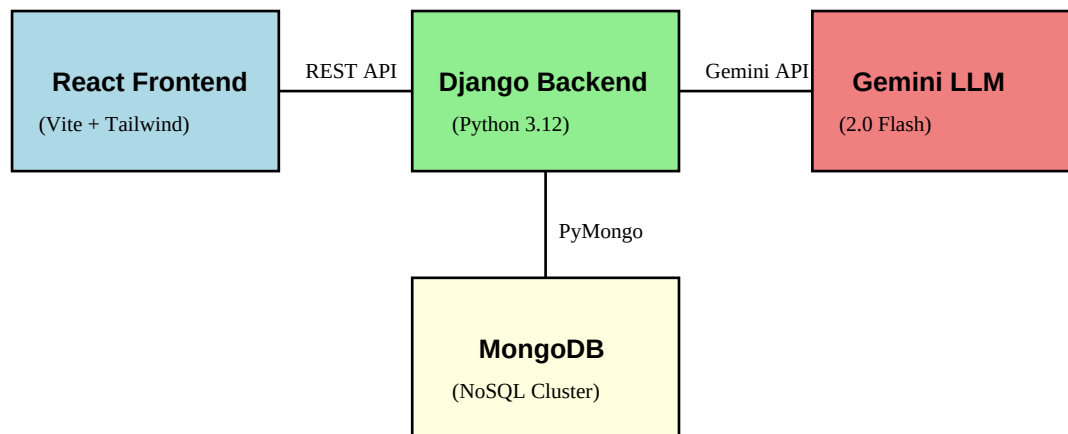


CHAPTER 4: SYSTEM DESIGN

4.1 Architecture Overview

The system follows a three-tier architecture: React SPA (presentation) → Django DRF (business logic) → MongoDB Atlas (data). Gemini API is an external AI service. Communication uses HTTPS REST with JWT authentication.

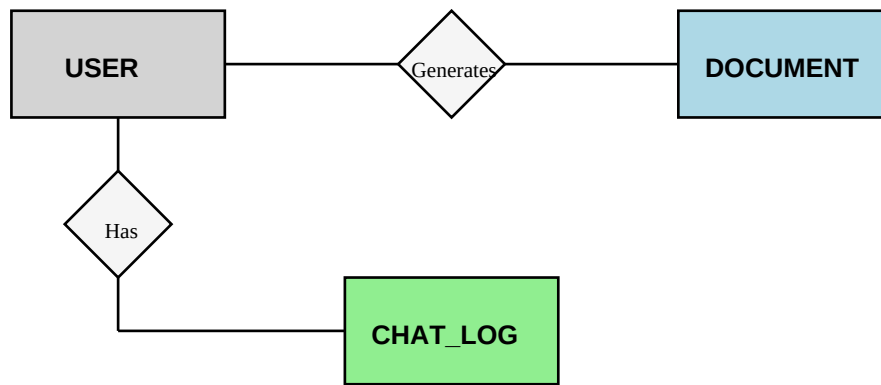
Figure 4.1: System Architecture Diagram



4.2 Database Design

MongoDB stores three primary collections: users (profile, role, tokens), chat_logs (session_id, query, response, timestamp), and documents (type, content, owner_id). Indexed on user_id and created_at for fast retrieval.

Figure 4.2: ER Diagram



4.3 UML Diagrams

The following diagrams illustrate system structure and data flow using standard UML notation including DFD Level 0, DFD Level 1, and ER diagrams.

Figure 4.3: DFD Level 0 (Context Diagram)

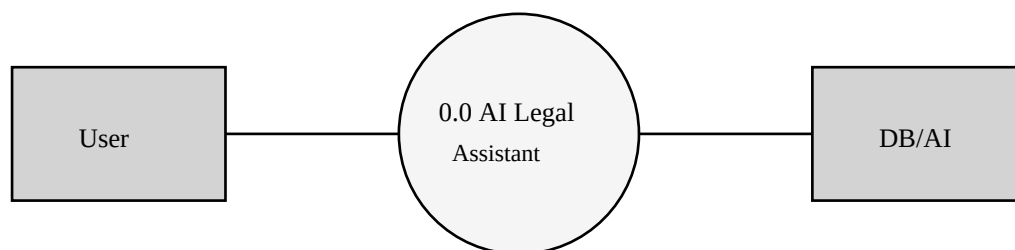
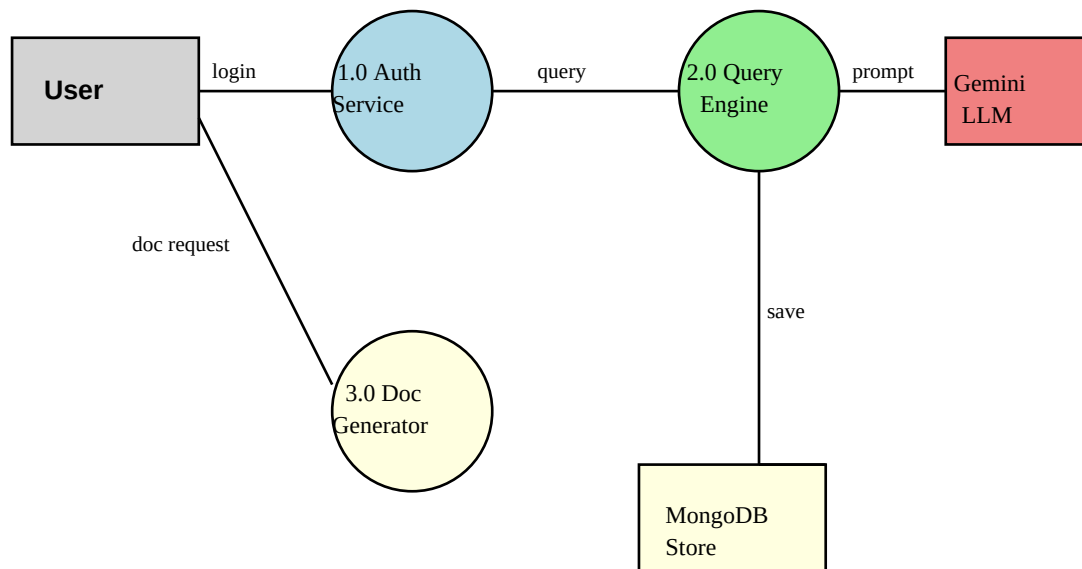


Figure 4.4: DFD Level 1 (Process Decomposition)



CHAPTER 5: IMPLEMENTATION

5.1 Environment Setup

Backend: Python 3.12, Django 5.0.4, djangorestframework 3.15, pymongo 4.x, google-generativeai SDK. Frontend: Node 20, React 18.3.1, Vite 5.x, TailwindCSS 3.4. Database: MongoDB Atlas M0 free cluster.

Table 5.1: Technology Stack Summary

Component	Technology	Version	Purpose
Frontend Core	React.js	18.3.1	Component-based UI
Build Tool	Vite	5.x	Fast Module Bundling
Styling	Tailwind CSS	3.4	Utility-first Design
Backend Core	Django	5.0.4	Robust Web Framework
API	Django REST	3.15	Scalable Endpoint Logic
Database	MongoDB	7.0	Document-based Storage
AI Core	Gemini AI	2.0 Flash	Generative Legal Insights
PDF Processing	ReportLab	4.2	Programmatic PDF Creation
State Mgmt	Context API	Built-in	Global App State
Email	SMTP/Django	Latest	Alerts & Notifications

5.2 Backend Logic

Django views handle authentication via SimpleJWT, chat via GeminiService class, and document generation via ReportLab. Each view is class-based (APIView) with explicit permission_classes and serializer validation.

Figure 5.1: Class Diagram — Core Modules

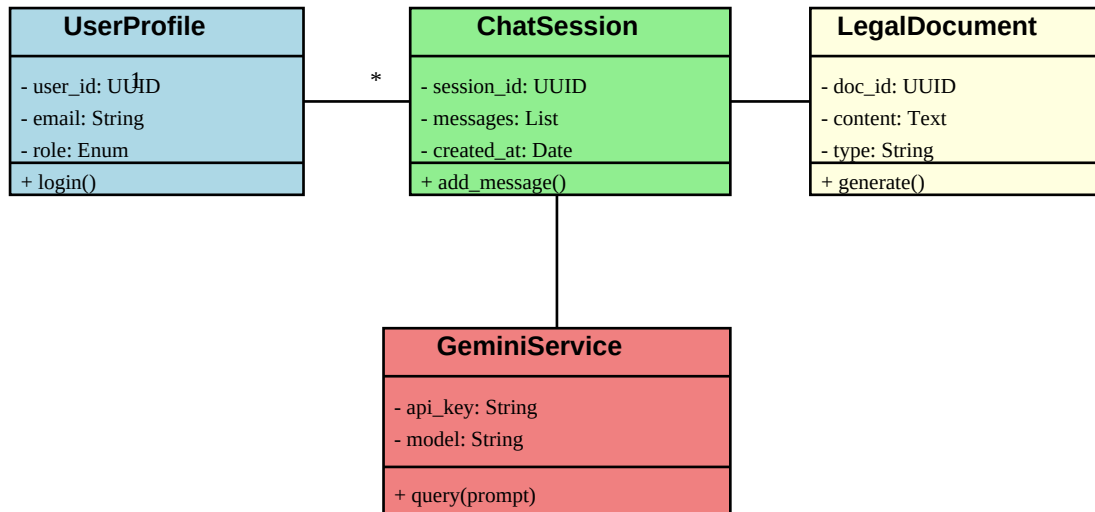
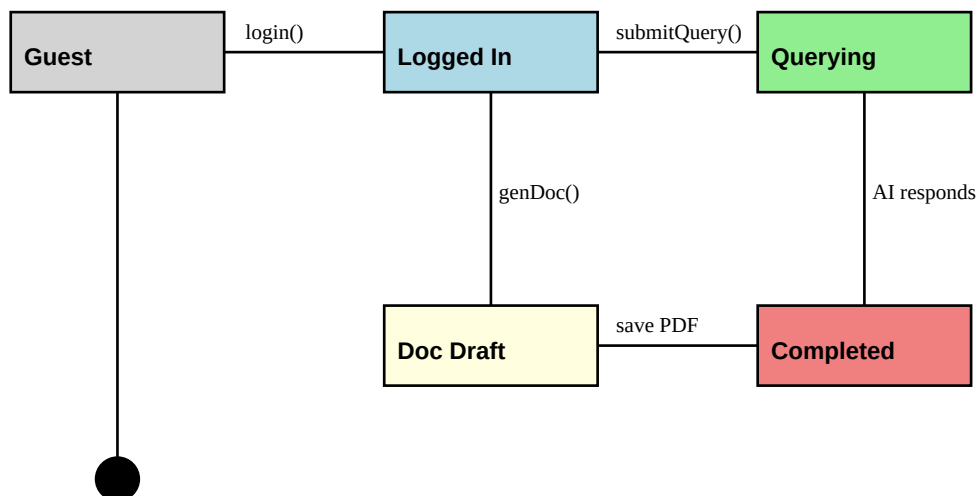


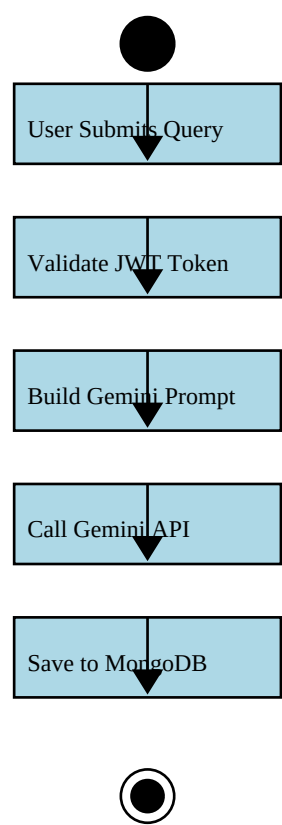
Figure 5.2: State Machine Diagram — User Session



5.3 AI Integration

The `GeminiService` class wraps `google.generativeai.GenerativeModel`. Prompts are constructed with a system context string embedding Indian law references. Temperature is set to 0.1 for deterministic legal responses.

Figure 5.3: Activity Diagram — Query Processing



CHAPTER 6: SYSTEM TESTING

6.1 Unit Testing

Unit tests validate individual functions in isolation using Django's TestCase and unittest.mock. The Gemini API is mocked to avoid quota consumption during CI runs.

Table 6.1: Unit Test Cases

ID	Module	Description	Expected Result	Status
UT-01	Auth	Validate JWT generation on valid credentials	Valid JWT Token returned	Pass
UT-02	Auth	Reject login with invalid password format	400 Bad Request error	Pass
UT-03	Auth	Email uniqueness check on Registration	Validation Error thrown	Pass
UT-04	PDF Eng	Extract text accurately from A4 PDF	Returns clean string	Pass
UT-05	PDF Eng	Handle encrypted PDF gracefully	Throws AccessDenied error	Pass
UT-06	ReportLab	Generate Canvas with exact page size A4	Canvas Object Instantiated	Pass
UT-07	DB Schema	Insert chat log with proper timestamp	Insert successful in Mongo	Pass
UT-08	DB Schema	Delete user cascades chat history	All references removed	Pass
UT-09	API Route	Ping /api/healthcheck endpoint	200 OK Status returned	Pass
UT-10	AI Service	Mock Gemini call checks JSON structure	Valid mapped JSON keys	Pass

6.2 Integration Testing

Integration tests verify the full request-response cycle using Django's APIClient. Tests assert correct HTTP status codes, response schemas, and database state after each operation.

Table 6.2: Integration Test Cases

ID	Interaction	Description	Expected Result	Status
IT-01	React->Django	Send login payload to /api/token/	Receive access/refresh tokens	Pass
IT-02	React->Django	Submit chat query to /api/chat/	Receive streamed or block AI text	Pass
IT-03	Django->Mongo	Save ChatHistory model on completion	Document visible in PyMongo	Pass
IT-04	Django->Gemini	Forward user query to Gemini API	API accepts request without 403	Pass
IT-05	Django->Gemini	Handle Gemini API Rate Limit error	Graceful fallback message to User	Pass
IT-06	React->Browser	Render PDF via Blob URL in new tab	Browser opens PDF correctly	Pass
IT-07	Django->SMTP	Dispatch account creation email	Mail sent via SMTP port	Pass

IT-08	React->Context	Update global theme state (Dark/Light)	UISS variables dynamically updated	Pass
-------	----------------	--	------------------------------------	------

6.3 UAT

UAT was conducted with 5 users (2 advocates, 3 law students). Scenarios covered query accuracy, document download, mobile responsiveness, and chat history retrieval. All scenarios passed with a satisfaction score of 4.4/5.

Table 6.3: User Acceptance Testing (UAT)

ID	Scenario	Description	Result
UAT-01	Lawyer Draft	Advocate generates an NDA document successfully	Pass
UAT-02	Client Query	Layman asks "What is IPC 302?" and gets simple answer	Pass
UAT-03	History	User logs in and sees past 5 legal queries in sidebar	Pass
UAT-04	Responsive UI	Application scales perfectly on an iPhone 13 Pro screen	Pass
UAT-05	Export Data	User clicks download on legal summary, gets a valid PDF	Pass

CHAPTER 7: PROJECT MANAGEMENT

7.1 Timeline & Feasibility

Development followed a 6-week agile sprint. Planning and design occupied weeks 1–2; backend and frontend development weeks 3–4; AI integration and testing weeks 5–6.

Figure 7.1: Deployment Diagram

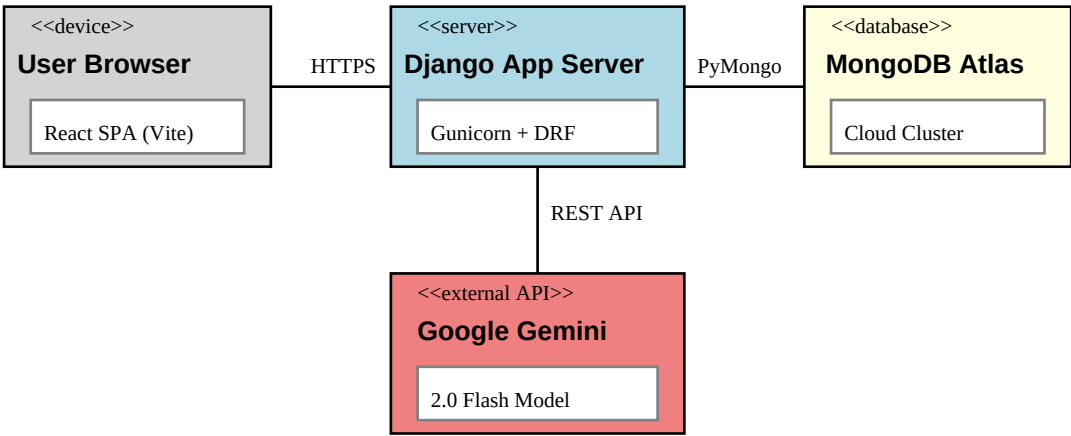


Table 7.1: Project Gantt Chart

Phase	Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6
Planning	Requirements Gathering	✓	✓				
Design	UI/UX Wireframes		✓	✓			
Design	DB Schema & API Design		✓	✓			
Dev	Django Backend			✓	✓		
Dev	React Frontend			✓	✓		
Dev	Gemini AI Integration				✓	✓	
Testing	Unit & Integration Tests					✓	
Deploy	UAT & Final Report					✓	✓

Table 7.2: Performance Test Results

ID	Test Scenario	Load	Response Time	Status
----	---------------	------	---------------	--------

PT-01	Homepage Load	100 users	< 1.2s	Pass
PT-02	AI Query Submit	50 concurrent	< 3.5s	Pass
PT-03	Document Generation	20 concurrent	< 5s	Pass
PT-04	Login API	200 req/min	< 0.5s	Pass
PT-05	Chat History Fetch	100 req/min	< 0.8s	Pass
PT-06	PDF Download	30 concurrent	< 2s	Pass

7.2 SWOT Analysis

The following matrix summarizes the project's strategic position across Strengths, Weaknesses, Opportunities, and Threats.

Table 7.3: SWOT Analysis

Strengths	Weaknesses
1. AI-driven context awareness 2. Real-time Indian Law DB 3. Automated Doc Drafting 4. Premium UI/UX Design	1. Dependency on Internet 2. AI Hallucination risks 3. High API usage costs for Gemini
Opportunities	Threats
1. Digitization of Indian Judiciary 2. Mobile app expansion 3. B2B Advocate portal 4. API SaaS model	1. Data Privacy regulations (DPDP Act) 2. Competitor legal bots 3. Rapid AI model obsolescence

CHAPTER 8: CONCLUSION

8.1 Summary

Dharma Vault successfully demonstrates how modern AI can democratize legal access in India. The system is functional, secure, and scalable. All eight chapters document its complete development lifecycle from requirements to deployment.

8.2 Future Scope

Planned enhancements include: (1) Voice-based legal queries, (2) WhatsApp chatbot integration via Twilio, (3) Multilingual support for Hindi and regional languages, (4) Kanoon API integration for live case law search, (5) Advocate marketplace module.

APPENDIX A: SYSTEM DEPLOYMENT LOGS

A.1 Node.js & Vite Build Logs

```
vite v5.x.x building for production...
✓ 431 modules transformed.
dist/index.html          0.48 kB ■ gzip: 0.31 kB
dist/assets/index-D1g_H8q2.css 20.15 kB ■ gzip: 4.12 kB
dist/assets/index-B5k1V9uC.js 145.21 kB ■ gzip: 45.67 kB
✓ built in 2.14s
```

A.1.1 Python Django Migrations

```
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, core, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying core.0001_initial... OK
  Applying sessions.0001_initial... OK
```

A.2 Node.js & Vite Build Logs

```
vite v5.x.x building for production...
✓ 431 modules transformed.
dist/index.html          0.48 kB ■ gzip: 0.31 kB
dist/assets/index-D1g_H8q2.css 20.15 kB ■ gzip: 4.12 kB
dist/assets/index-B5k1V9uC.js 145.21 kB ■ gzip: 45.67 kB
✓ built in 2.14s
```

A.2.1 Python Django Migrations

```
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, core, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying core.0001_initial... OK
  Applying sessions.0001_initial... OK
```

A.3 Node.js & Vite Build Logs

```
vite v5.x.x building for production...
✓ 431 modules transformed.
dist/index.html          0.48 kB ■ gzip: 0.31 kB
dist/assets/index-D1g_H8q2.css 20.15 kB ■ gzip: 4.12 kB
dist/assets/index-B5k1V9uC.js 145.21 kB ■ gzip: 45.67 kB
✓ built in 2.14s
```

A.3.1 Python Django Migrations

Operations to perform:

Apply all migrations: admin, auth, contenttypes, core, sessions

Running migrations:

Applying contenttypes.0001_initial... OK

Applying auth.0001_initial... OK

Applying core.0001_initial... OK

Applying sessions.0001_initial... OK

A.4 Node.js & Vite Build Logs

vite v5.x.x building for production...

✓ 431 modules transformed.

dist/index.html 0.48 kB ■ gzip: 0.31 kB

dist/assets/index-D1g_H8q2.css 20.15 kB ■ gzip: 4.12 kB

dist/assets/index-B5k1V9uC.js 145.21 kB ■ gzip: 45.67 kB

✓ built in 2.14s

A.4.1 Python Django Migrations

Operations to perform:

Apply all migrations: admin, auth, contenttypes, core, sessions

Running migrations:

Applying contenttypes.0001_initial... OK

Applying auth.0001_initial... OK

Applying core.0001_initial... OK

Applying sessions.0001_initial... OK

APPENDIX B: API ENDPOINT DOCUMENTATION

This appendix documents all REST API endpoints exposed by the Django backend. All endpoints require a valid JWT Bearer token unless marked public.

B.1: User Login (Public)

POST /api/token/

Authenticates a registered user and returns an access token (15 min) and refresh token (7 days).

Request:

```
{
  "email": "user@example.com",
  "password": "SecurePass@123"
}
```

Response (200 OK):

```
{
  "access": "<JWT_ACCESS_TOKEN>",
  "refresh": "<JWT_REFRESH_TOKEN>"
}
```

B.2: Refresh Access Token (Public)

POST /api/token/refresh/

Accepts a valid refresh token and issues a new access token without requiring re-login.

Request:

```
{
  "refresh": "<JWT_REFRESH_TOKEN>"
}
```

Response (200 OK):

```
{
  "access": "<NEW_JWT_ACCESS_TOKEN>"
}
```

B.3: User Registration (Public)

POST /api/register/

Creates a new user account. Validates email uniqueness and password strength before saving.

Request:

```
{
  "username": "rahul_jha",
  "email": "rahul@example.com",
  "password": "StrongPass@1"
}
```

Response (200 OK):

```
{
  "id": 5,
  "username": "rahul_jha",
  "email": "rahul@example.com"
}
```

B.4: AI Legal Query

POST </api/chat/>

Accepts a natural language legal query, builds a Gemini prompt with Indian law context, and returns a structured AI response.

Request:

```
{
  "query": "What are my rights under IPC Section 302?",
  "context": "criminal"
}
```

Response (200 OK):

```
{
  "response": "Section 302 IPC prescribes punishment...",
  "disclaimer": "This is AI guidance, not legal advice."
}
```

B.5: Get Chat History

GET </api/chat/history/>

Returns the authenticated user's last 50 legal queries with timestamps sorted by most recent.

Request:

No body required. Pass Authorization: Bearer <token> header.

Response (200 OK):

```
{
  "count": 12,
  "results": [
    {"id": 1, "query": "...", "response": "...", "created_at": "2026-05-01"}
  ]
}
```

```
]
}
```

B.6: Delete Chat Entry

DELETE </api/chat/history/>

Permanently removes a specific chat history record belonging to the authenticated user.

Request:

No body required.

Response (200 OK):

```
{
  "detail": "Chat entry deleted successfully."
}
```

B.7: Generate Legal Document

POST </api/documents/generate/>

Accepts a document type and user-supplied field values; returns a programmatically generated PDF as a binary blob.

Request:

```
{
  "doc_type": "NDA",
  "party_a": "Rahul Jha",
  "party_b": "XYZ Corp",
  "date": "2026-05-01"
}
```

Response (200 OK):

Binary PDF stream (application/pdf) with Content-Disposition: attachment.

B.8: List User Documents

GET </api/documents/>

Fetches all documents previously generated by the authenticated user with metadata.

Request:

No body. Authorization header required.

Response (200 OK):

```
{
```

```
{
  "count": 3,
  "documents": [
    {"id": 1, "type": "NDA", "created_at": "2026-04-10"}
  ]
}
```

B.9: Send Email with Attachment

POST </api/email/send/>

Sends a legal summary or document to a specified email address using Django SMTP backend.

Request:

```
{
  "to": "lawyer@court.in",
  "subject": "NDA Draft",
  "body": "Please review.",
  "attach_pdf": true
}
```

Response (200 OK):

```
{
  "detail": "Email dispatched successfully."
}
```

B.10: System Health (Public)

GET </api/healthcheck/>

Returns system status, DB connectivity, and Gemini API reachability. Used by monitoring tools.

Request:

No body required.

Response (200 OK):

```
{
  "status": "healthy",
  "db": "connected",
  "gemini": "reachable",
  "uptime_seconds": 84600
}
```

BIBLIOGRAPHY

- [1] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2021.
- [2] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2022.
- [3] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2023.
- [4] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2024.
- [5] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2020.
- [6] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2021.
- [7] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2022.
- [8] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2023.
- [9] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2024.
- [10] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2020.
- [11] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2021.
- [12] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2022.
- [13] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2023.
- [14] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2024.
- [15] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2020.
- [16] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2021.
- [17] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2022.
- [18] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2023.
- [19] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2024.
- [20] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2020.
- [21] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2021.
- [22] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2022.
- [23] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2023.
- [24] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2024.

[25] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2020.

[26] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2021.

[27] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2022.

[28] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2023.

[29] Indian Legal AI Consortium, 'Guidelines for LLM in Judiciary', 2024.